SIMULAÇÃO COMPUTACIONAL APLICADA À ENGENHARIA NUCLEAR

05 NOV 26

MAJ KREUSCH

GEANT4

FUNDAMENTOS E APLICAÇÃO PRÁTICA

PPGEN
Programa de Pós-Graduação em Engenharia Nuclear do IME

Objetivos

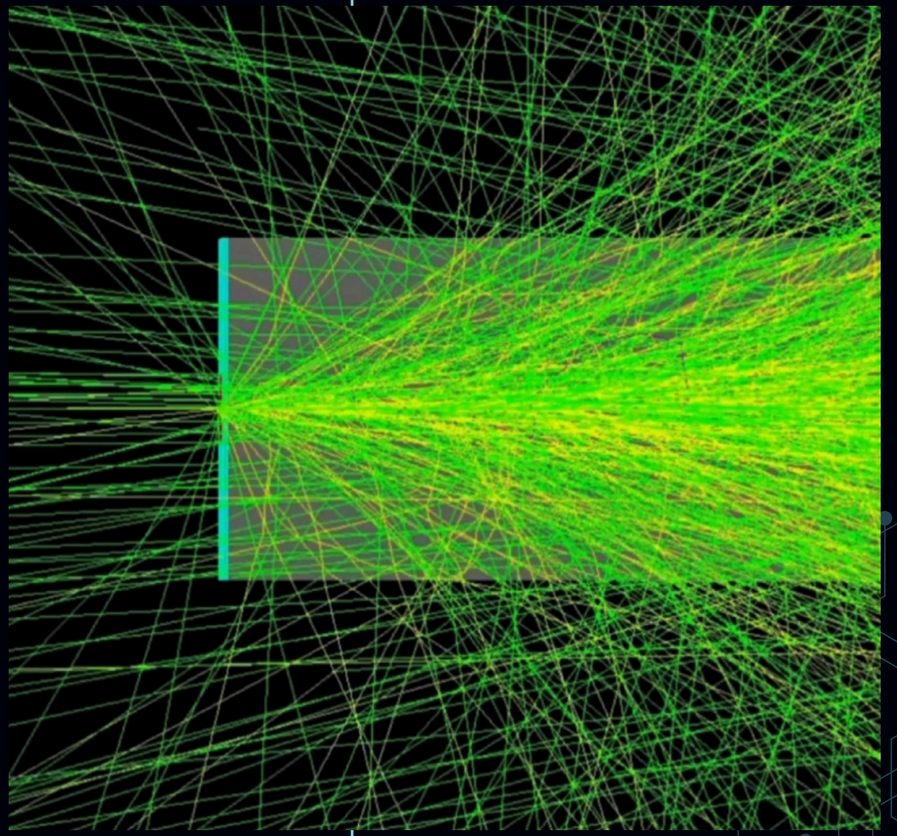
01. Compreender a filosofia do Geant4

02. Identificar os blocos principais de uma aplicação Geant4

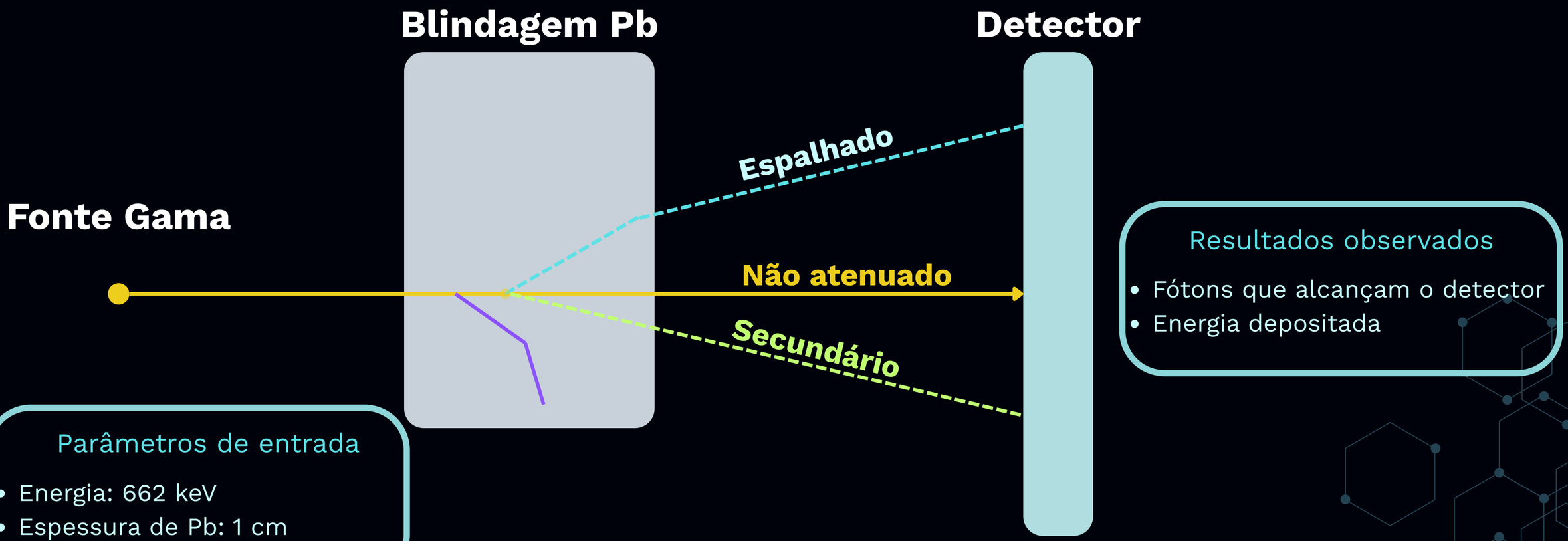
03. Estabelecer um comparativo crítico entre o Geant4 e o MCNP5

04. Assimilar a estrutura básica de um código em Geant4





Problema exemplo



Parâmetros de entrada

- Energia: 662 keV
- Espessura de Pb: 1 cm

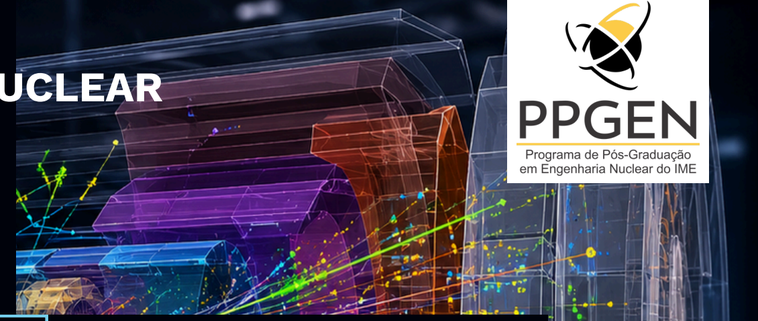
Resultados observados

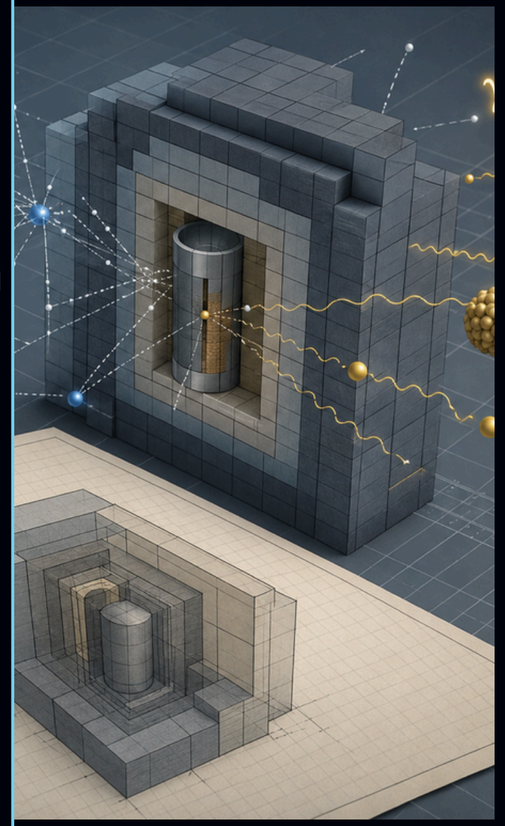
- Fótons que alcançam o detector
- Energia depositada

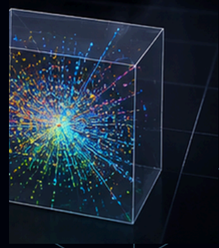
Como podemos construir e analisar essa simulação no MCNP?

Do MCNP5 ao Geant4

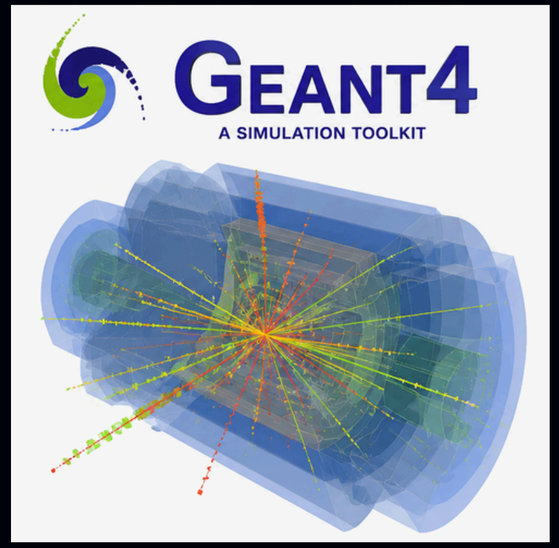
MCNP5		Geant4
Arquivo de entrada	→	Aplicação C++
Células	→	Volumes } G4DetectorConstruction
Superfícies	→	Sólidos } G4DetectorConstruction
Cartão de materiais (M)	→	G4Material
Definição de fonte (SDEF)	→	PrimaryGeneratorAction
Modo de transporte (MODE)	→	Physics List
Tallies (F1, F4, F6, ...)	→	Scorers / SensitiveDetector
Histórias (NPS)	→	Eventos (/run/beamOn)







O que é o Geant4?



- *Toolkit* Monte Carlo
- Simula a passagem de partículas pela matéria
- Desenvolvido em C++ com arquitetura orientada a objetos



- Permite construir aplicações específicas de simulação
- Muito usado em física nuclear, detectores, medicina, espaço e proteção radiológica

Origem e filosofia

Origem e filosofia

Desenvolvido no CERN no final da década de 1990

1994
Início do projeto Geant4 no CERN

1999
Primeira versão pública

2003
Geant4 torna-se um projeto de código aberto

Hoje
Utilizado mundialmente na ciência e na indústria

Criado para responder às necessidades de simulações cada vez mais precisas de detectores complexos e ambientes de radiação.



Filosofia do Geant4

Versatilidade
Aplicável a uma ampla gama de áreas: física de altas energias, medicina, espaço, indústria e mais.

Modularidade
Arquitetura modular que permite fácil extensão e personalização.

Precisão
Modelos físicos avançados para descrever o transporte de partículas e suas interações.

Código aberto
Transparência, colaboração evolução contínua da comunidade científica global.

Colaboração
Desenvolvido e mantido por uma colaboração internacional ativa.

Fluxo de trabalho

Problema físico

↳ **Geometria e materiais**

↳ **Physics list**

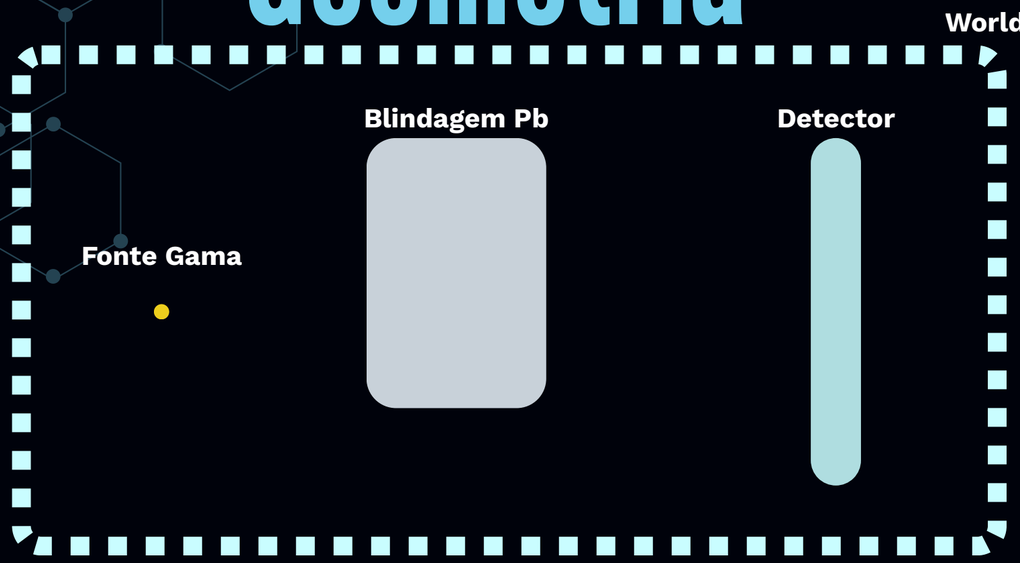
↳ **Fonte primária**

↳ **Scoring/detectores**

↳ **Execução dos eventos**

↳ **Análise dos resultados**

Geometria



World: 40 × 40 × 40 cm

Pb: 6 × 6 × 1 cm (referência)

Detector Si: 6 × 6 cm, espessura 1 mm

```
auto solidPb = new G4Box("Pb", 3*cm, 3*cm, 0.5*cm);
auto logicPb = new G4LogicalVolume(solidPb, lead,
"Pb");
new G4PVPlacement(nullptr, {}, logicPb, "Pb",
                  logicWorld, false, 0);
```

DetectorConstruction.cc

Hierarquia de Construção

Sólido

Define a forma geométrica: caixa, cilindro, cone, etc.

G4Box

Volume lógico

Associa forma + material

G4LogicalVolume

Volume físico

Posiciona o volume no espaço

G4PVPlacement

Materiais

Isótopos
- Número atômico - Z
- Número de núcleons - N
- Número de massa - A

Elementos
- Elemento químico
- Número atômico
- Composição isotópica

Materiais
- Composição química
- Densidade
- Estado físico (T, P)

Materiais pré-definidos

G4NistManager

DetectorConstruction.cc

```
auto nist = G4NistManager::Instance();
G4Material* air = nist-> FindOrBuildMaterial("G4_AIR");
G4Material* lead = nist-> FindOrBuildMaterial("G4_Pb");
```

Lista de física

O que é definido?

- **Partículas**
  Fótons, elétrons, pósitrons, nêutrons, prótons, íons, múons...
- **Processos**
  Compton, fotoelétrico, ionização, reações nucleares...
- **Modelos e parâmetros**
  Modelos físicos, faixas de validade, cortes de produção...

Listas de referência

- **FTFP_BERT**
  Uso geral, altas energias + cascata hadrônica
- **Shielding**
  Aplicações de blindagem e transporte de nêutrons
- **Livermore/Penelope**
  Eletromagnética de baixa energia

main.cc

```
#include "FTFP_BERT.hh"

runManager->SetUserInitialization(new FTFP_BERT());
```

Fonte primária

Parâmetros da fonte

- **Tipo de Partícula**
- **Energia Inicial**
- **Posição de Emissão**
- **Direção de Emissão**

Geradores disponíveis

G4ParticleGun

Fonte simples:

- partícula monoenergética
- posição fixa
- direção fixa
- feixe colimado
- fonte pontual simplificada

G4GeneralParticleSource

Distribuições mais realistas:

- fonte volumétrica
- fonte superficial
- distribuições angular e energética
- múltiplas fontes
- fonte extensa

PrimaryGeneratorAction.cc

```
fParticleGun = new G4ParticleGun(1);
fParticleGun->SetParticleDefinition(gamma);
fParticleGun->SetParticleEnergy(662*keV);
fParticleGun->SetParticlePosition({0, 0, -10*cm});
fParticleGun->SetParticleMomentumDirection({0, 0, 1});
```

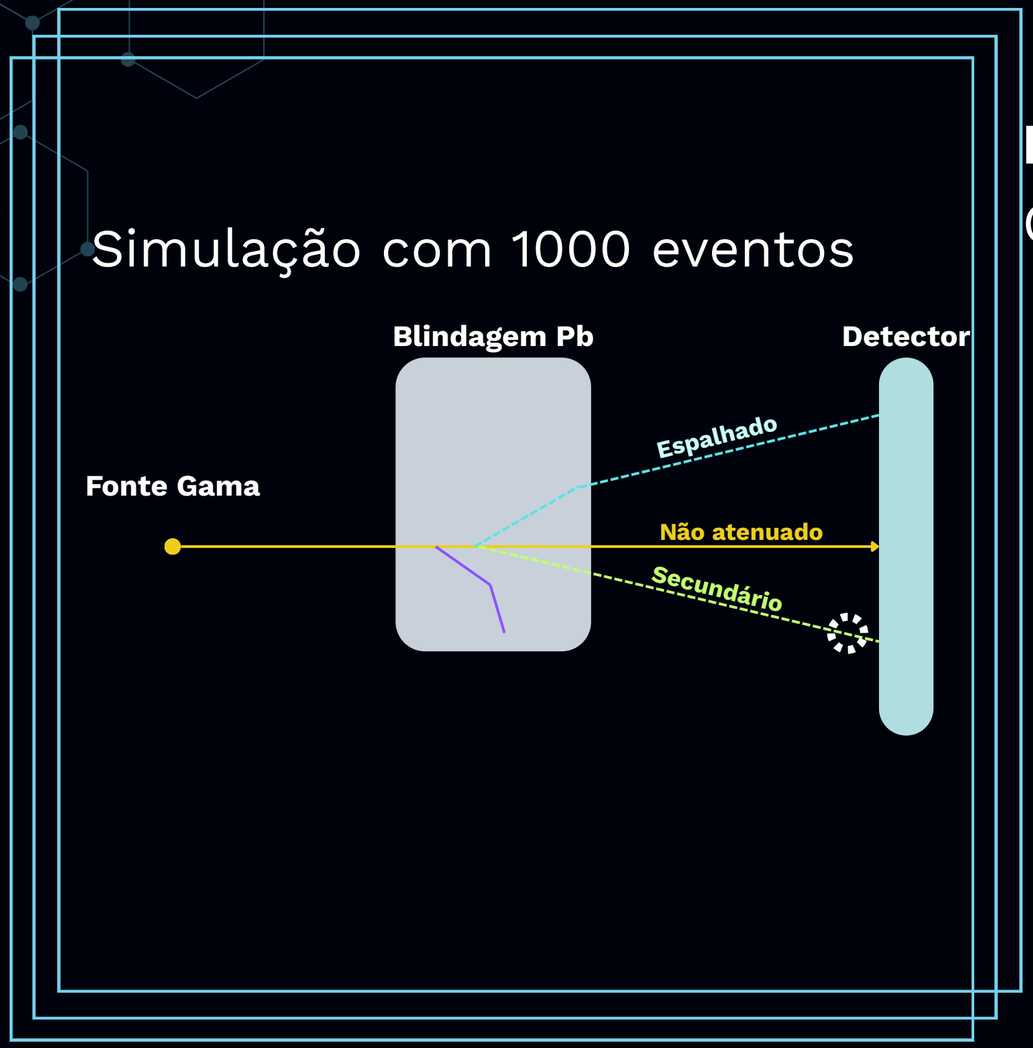
run.mac

Macros/comandos

Ajustes simples durante a execução da simulação

```
/gun/particle gamma
/gun/energy 1 MeV
/gun/position 0 0 -10 cm
/gun/direction 0 0 1
```

Transporte no Geant4



Run

Conjunto de eventos simulados

↳ **Evento**

Uma partícula primária + todos secundários gerados

↳ **Track**

Trajetória de uma partícula individual

↳ **Step**

Pequeno trecho da trajetória

run.mac

Macros/comandos

/run/beamOn 1000

Ações do usuário

Definem comportamentos em pontos específicos

PrimaryGeneratorAction

Início de cada evento

Gera as partículas primárias

RunAction

Início e fim do run

Inicializa e consolida resultados

EventAction

Início e fim de cada evento

Reinicia e acumula dados por evento

SteppingAction

A cada step

Inspeciona o transporte e registra ocorrências

Exemplo: medir a chegada de fótons ao detector

SteppingAction

Identifica a entrada de um fóton no detector

→

EventAction

Registra número de fótons detectados

→

RunAction

Acumula os eventos e apresenta o resultado

ActionInitialization.cc

```
void ActionInitialization::Build() const
{
 SetUserAction(new
PrimaryGeneratorAction());
 SetUserAction(new RunAction());
 SetUserAction(new EventAction());
 SetUserAction(new SteppingAction());
}
```

Scoring

Transforma informações do tracking em resultados físicos.

Pode ser feito por:

UserActions	Sensitive Detectors	Primitive Scorers
SteppingAction e EventAction Lógica personalizada (grandezas e condições específicas)	G4VSensitiveDetector Informações dos steps em volumes sensíveis	G4MultiFunctionalDetector Grandezas pré-definidas: energia, dose, fluxo...

SteppingAction.cc

```
void
SteppingAction::UserSteppingAction(con
st G4Step* step)
{
 auto volume = step->GetPreStepPoint()
 ->GetTouchableHandle()
 ->GetVolume()
 ->GetLogicalVolume();
 if (volume->GetName() != "Detector")
return;
...
```

```
...

 auto edep = step-
>GetTotalEnergyDeposit();

 if (edep > 0.) {
 fEventAction->AddEdep(edep);
 }
}
```

Saída e análise

Os resultados podem ser organizados por:

- EventAction: acumula dados por evento
- RunAction: consolida dados do run
- G4AnalysisManager: salva histogramas, ntuples e arquivos
- Arquivos de saída: .csv, .root, .hdf5, .txt

```
auto analysis = G4AnalysisManager::Instance();

analysis->CreateNtuple("Hits", "Detector data");
analysis->CreateNtupleDColumn("Edep");
analysis->CreateNtupleDColumn("Time");
analysis->FinishNtuple();
```

Geant4 x MCNP5

MCNP5

Vantagens

- Definição por arquivo de entrada
- Tallies e técnicas consolidadadas
- Aplicações em blindagem, transporte e criticalidade

Limitações

- Menor liberdade de personalização
- Extensões limitadas

FERRAMENTAS COMPLEMENTARES

Geant4

Vantagens

- Flexibilidade
- Personalização do acompanhamento (tracks e steps)
- Integração com aplicações e bibliotecas

Limitações

- Maior curva de aprendizagem
- Exige implementação, testes e manutenção da aplicação

Possibilidade de uso combinado

Arquitetura geral

Estrutura de diretórios

```
Geant4Demo/
├── main.cc
├── CMakeLists.txt
├── run.mac
├── vis.mac
├── include/
│   ├── DetectorConstruction.hh
│   ├── PrimaryGeneratorAction.hh
│   ├── ActionInitialization.hh
│   ├── RunAction.hh
│   ├── EventAction.hh
│   └── SteppingAction.hh
├── src/
│   ├── DetectorConstruction.cc
│   ├── PrimaryGeneratorAction.cc
│   ├── ActionInitialization.cc
│   ├── RunAction.cc
│   ├── EventAction.cc
│   └── SteppingAction.cc
└── build/
```

main.cc

Ponto de entrad
os componentes

include/ e src/

Declarações (.hh
classes da aplic

CMakeLists.txt

Configura a co
bibliotecas Gear

Arquivos .mac

Comandos para
visualização

Código fonte

Define a aplicação

Compilação

Produz o executável

Execução

Comandos e macros

Demonstração - compilando e rodando

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(Geant4Demo)

find_package(Geant4 REQUIRED ui_all vis_all)
include(${Geant4_USE_FILE})

include_directories(${PROJECT_SOURCE_DIR}/include)

file(GLOB sources ${PROJECT_SOURCE_DIR}/src/*.cc)
file(GLOB headers ${PROJECT_SOURCE_DIR}/include/*.hh)

add_executable(Geant4Demo main.cc ${sources} ${headers})
target_link_libraries(Geant4Demo ${Geant4_LIBRARIES})
```

Comandos no terminal

```
mkdir build
cd build
cmake ..
make -j$(nproc)

./Geant4Demo
```

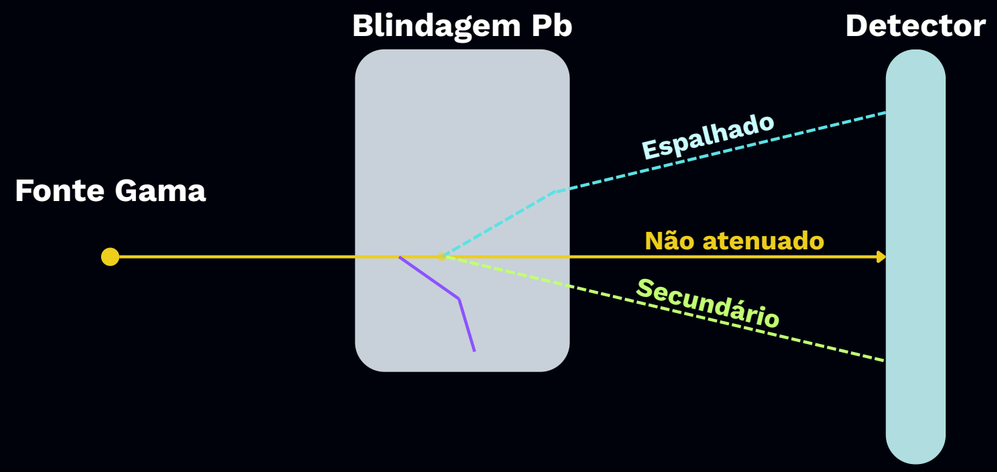
Rodando com macros

run.mac

```
/run/initialize
/gun/particle gamma
/gun/energy 1 MeV
/gun/position 0 0 -10 cm
/gun/direction 0 0 1
/run/beamOn 1000
```

```
./Geant4Demo run.mac
```

Demonstração



Fonte

- Partícula: fóton gama
- Energia: 662 keV
- Posição: z=-10 cm
- Direção: +z

Geometria

- Pb: 6 x 6 cm
- Espessura: 1 cm
- Si: 6 cm x 6 cm x 1 mm

Resultados

- Fraçao de eventos
- Energia depositada no Si
- Incertezas estatísticas

Preparação para a aula prática

https://github.com/diogeneskreusch/Geant4_Aula


Instituto Militar de Engenharia (IME)
Programa de Pós-Graduação em Engenharia Nuclear (PPGEN)
Simulação Computacional Aplicada à Engenharia Nuclear

Instalação e primeiros passos com o Geant4

- Instalar o WSL 2 e o Ubuntu 24.04 LTS
- Instalar e configurar o Geant4
- Baixar o projeto Geant4Demo do GitHub
- Compilar a aplicação com CMake
- Executar teste inicial



Dúvidas?

Maj Kreusch

E-mail :

kreusch.diogenes@ime.eb.br

